

- Create a class `Person` whose constructor takes `age` as an argument. Raise a `ValueError` if the `age` is less than 0.
- Write a function named `find_length(obj)` that uses a loop to calculate the length of the given object without using the built-in `len()` function. The function should return the calculated length if the object is iterable. If a non-iterable object such as an integer is passed, the function should raise and handle a `TypeError`, and print an appropriate error message explaining what happens when an integer is sent as input.
- Create a class `Student` with an attribute `marks`. Implement a method `set_marks(marks)` that raises a `ValueError` if `marks` are not in the range 0 to 100.
- Create a custom exception named `InvalidAgeError`. Create a class `Voter` with a method `check_eligibility(age)` that raises this exception if `age` is less than 18.
- Create a class `BankAccount` with an attribute `balance`. Implement a method `withdraw(amount)` that raises an exception if the withdrawal amount is greater than the available balance.
- Create a class `PasswordValidator` with a method `validate(password)`. Raise an exception if the password length is less than 8 characters.
- Create a class `UserInput` with a method `get_integer(value)`. Handle `ValueError` and `TypeError` using separate `except` blocks.
- Create a base class `Shape` with a method `area()` that raises `NotImplementedError`. Create a child class `Rectangle` that overrides and implements the `area` method.
- Create a class `Service` with a method that calls another method which raises an exception. Catch and handle the exception in the `Service` class.
- Create a class `Transaction` with a method `process()` that uses `try`, `except`, and `finally` blocks to ensure a cleanup message is always printed.
- Create a class `LoginSystem` with a method `login(password)` that raises an exception for an incorrect password and handles the exception outside the class.